Task 1

```r
#Start Section/Chunk
#installing packages
#install.packages(c("data.table", "readr", "ggplot2", "stringr"), dependencies = TRUE)

#loading packages
library(data.table)
library(readr)
library(ggplot2)
library(stringr)

# getwd checks folder name, list.files checks files in folder and their names
getwd()
```

```
## [1] "/cloud/project/quantium_forage_project_folder"
```

```r
list.files()
```

```
## [1] "PDF_Example_Code_Given.R"   "QVI_purchase_behaviour.csv"
## [3] "QVI_transaction_data.csv"   "Task1_files"
## [5] "Task1.knit.md"              "Task1.pdf"
## [7] "Task1.Rmd"
```

```r
# load data. create name/variable to store transaction and customer data set after loading it
transactionData <- fread("QVI_transaction_data.csv")
customerData <- fread("QVI_purchase_behaviour.csv")

# confirm the datasets are loaded. shows dimensions IE number of columns and rows
dim(transactionData)
```

```
## [1] 264836      8
```

```r
dim(customerData)
```

```
## [1] 72637     3
```

```r
#Data Cleaning Chunk/ Section
# quick structure checks. checking column names + types
### commented out after checking



names(transactionData)
```

```
## [1] "DATE"           "STORE_NBR"      "LYLTY_CARD_NBR" "TXN_ID"
## [5] "PROD_NBR"       "PROD_NAME"      "PROD_QTY"       "TOT_SALES"
```

```r
# str(transactionData)

names(customerData)
```

```
## [1] "LYLTY_CARD_NBR"   "LIFESTAGE"        "PREMIUM_CUSTOMER"
```

```r
# str(customerData)
#result: because date is INT, need to change to DATE

# stats for quantities of chips purchased IE min, max, median, 1st quartile (25th percentile) and 3rd q
#PROD_QTY shows how many chip packages bought in transaction
summary(transactionData$PROD_QTY)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   2.000   2.000   1.907   2.000 200.000
```

```r
# stats for sales of chips IE min, max, median, 1st quartile (25th percentile) and 3rd quartile (75th p
#TOT_SALES shows dollar amount for that line item
summary(transactionData$TOT_SALES)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.500   5.400   7.400   7.304   9.200 650.000
```

```r
# counting missing values (NA counts by column)
colSums(is.na(transactionData))
```

```
##           DATE      STORE_NBR LYLTY_CARD_NBR         TXN_ID       PROD_NBR
##              0              0              0              0              0
##      PROD_NAME       PROD_QTY      TOT_SALES
##              0              0              0
```

```r
colSums(is.na(customerData))
```

```
##   LYLTY_CARD_NBR        LIFESTAGE PREMIUM_CUSTOMER
##                0                0                0
```

```r
#result: all 0s, so it is good



#FIX DATES
# change Date from INT into DATE
#1899-12-30 is origin date used by Excel-style date numbers
transactionData$DATE <- as.Date(transactionData$DATE, origin = "1899-12-30")

# confirm conversion worked
summary(transactionData$DATE)
```

```
##         Min.      1st Qu.       Median         Mean      3rd Qu.         Max.
## "2018-07-01" "2018-09-30" "2018-12-30" "2018-12-30" "2019-03-31" "2019-06-30"
```

```r
#CHECK TO MAKE SURE ALL PRODUCTS ARE CHIPS
#show 20 product names
head(transactionData$PROD_NAME, 20)
```

```
##  [1] "Natural Chip        Compny SeaSalt175g"
##  [2] "CCs Nacho Cheese    175g"
##  [3] "Smiths Crinkle Cut  Chips Chicken 170g"
##  [4] "Smiths Chip Thinly  S/Cream&Onion 175g"
##  [5] "Kettle Tortilla ChpsHny&Jlpno Chili 150g"
##  [6] "Old El Paso Salsa   Dip Tomato Mild 300g"
##  [7] "Smiths Crinkle Chips Salt & Vinegar 330g"
##  [8] "Grain Waves         Sweet Chilli 210g"
##  [9] "Doritos Corn Chip Mexican Jalapeno 150g"
```

```
## [10] "Grain Waves Sour    Cream&Chives 210G"
## [11] "Smiths Crinkle Chips Salt & Vinegar 330g"
## [12] "Kettle Sensations   Siracha Lime 150g"
## [13] "Twisties Cheese     270g"
## [14] "WW Crinkle Cut      Chicken 175g"
## [15] "Thins Chips Light&  Tangy 175g"
## [16] "CCs Original 175g"
## [17] "Burger Rings 220g"
## [18] "NCC Sour Cream &    Garden Chives 175g"
## [19] "Doritos Corn Chip Southern Chicken 150g"
## [20] "Cheezels Cheese Box 125g"
```

```r
#Result: found product named "Old El Paso Salsa", meaning i have to find and remove any products that a

# make product names lowercase (easier to search)
prod_lower <- tolower(transactionData$PROD_NAME)

# define non-chip keywords
non_chip_terms <- c("salsa", "dip")

# mark rows that contain any of these words
transactionData$NON_CHIP <- grepl(paste(non_chip_terms, collapse="|"), prod_lower)

# see how many rows will be removed
sum(transactionData$NON_CHIP)
```

```
## [1] 19532
```

```r
# show unique product names that are flagged as NON_CHIP (limit 20 so result is not too long)
head(unique(transactionData$PROD_NAME[transactionData$NON_CHIP])[1:20])
```

```
## [1] "Old El Paso Salsa   Dip Tomato Mild 300g"
## [2] "Red Rock Deli SR    Salsa & Mzzrlla 150g"
## [3] "Smiths Crinkle Cut  French OnionDip 150g"
## [4] "Smiths Crinkle Cut  Tomato Salsa 150g"
## [5] "Doritos Salsa       Medium 300g"
## [6] "Old El Paso Salsa   Dip Chnky Tom Ht300g"
```

```r
prod_lower <- tolower(transactionData$PROD_NAME)

# names that contain "dip" (changed to only 10 to reduce pdf size)
head(unique(transactionData$PROD_NAME[grepl("dip", prod_lower)])[1:10])
```

```
## [1] "Old El Paso Salsa   Dip Tomato Mild 300g"
## [2] "Smiths Crinkle Cut  French OnionDip 150g"
## [3] "Old El Paso Salsa   Dip Chnky Tom Ht300g"
## [4] "Old El Paso Salsa   Dip Tomato Med 300g"
## [5] NA
## [6] NA
```

```r
# names that contain "salsa" but NOT "dip" (possible salsa-flavored chips)(changed to only 10 to reduce
head(unique(transactionData$PROD_NAME[grepl("salsa", prod_lower) & !grepl("dip", prod_lower)])[1:10])
```

```
## [1] "Red Rock Deli SR    Salsa & Mzzrlla 150g"
## [2] "Smiths Crinkle Cut  Tomato Salsa 150g"
## [3] "Doritos Salsa       Medium 300g"
## [4] "Woolworths Mild     Salsa 300g"
```

```
## [5] "Woolworths Medium   Salsa 300g"
## [6] "Doritos Salsa Mild  300g"
```

```r
#remove NA from results
prod_lower <- tolower(transactionData$PROD_NAME)

salsa_not_dip_names <- unique(transactionData$PROD_NAME[
  !is.na(prod_lower) &
  grepl("salsa", prod_lower) &
  !grepl("dip", prod_lower)
])

salsa_not_dip_names
```

```
## [1] "Red Rock Deli SR    Salsa & Mzzrlla 150g"
## [2] "Smiths Crinkle Cut  Tomato Salsa 150g"
## [3] "Doritos Salsa       Medium 300g"
## [4] "Woolworths Mild     Salsa 300g"
## [5] "Woolworths Medium   Salsa 300g"
## [6] "Doritos Salsa Mild  300g"
```

```r
#result: these are mostly salsa chips, not dip. need to remove only the oners that are dip


# lowercase version for matching
prod_lower <- tolower(transactionData$PROD_NAME)

#flag anything with the word "dip" or "dips" in the product name
#The pattern \\bdips?\\b means match the word dip or dips:
#dip is the word
#s? means "the s is optional" (so dip or dips)
#\\b means "word boundary" so it matches the whole word, not part of another word

is_dip_word <- !is.na(prod_lower) & grepl("\\bdips?\\b", prod_lower)

#flag the Woolworths salsa items that are likely dip
is_woolworths_salsa <- !is.na(prod_lower) &
  grepl("woolworths", prod_lower) &
  grepl("salsa", prod_lower)

# combine both rules
transactionData$NON_CHIP <- is_dip_word | is_woolworths_salsa

# review what will be removed (first 50 unique names) (changed to 10 later on to reduce PDF size)
head(unique(transactionData$PROD_NAME[transactionData$NON_CHIP])[1:10])
```

```
## [1] "Old El Paso Salsa   Dip Tomato Mild 300g"
## [2] "Old El Paso Salsa   Dip Chnky Tom Ht300g"
## [3] "Woolworths Mild     Salsa 300g"
## [4] "Old El Paso Salsa   Dip Tomato Med 300g"
## [5] "Woolworths Medium   Salsa 300g"
## [6] NA
```

```r
# remove those rows
transactionData <- transactionData[transactionData$NON_CHIP == FALSE, ]
```

```r
# clean up helper column
transactionData$NON_CHIP <- NULL
```

```r
#checking after cleaning
dim(transactionData)
```

```
## [1] 252591      8
```

```r
summary(transactionData$PROD_QTY)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   2.000   2.000   1.907   2.000 200.000
```

```r
summary(transactionData$TOT_SALES)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.700   5.400   7.400   7.266   8.800 650.000
```

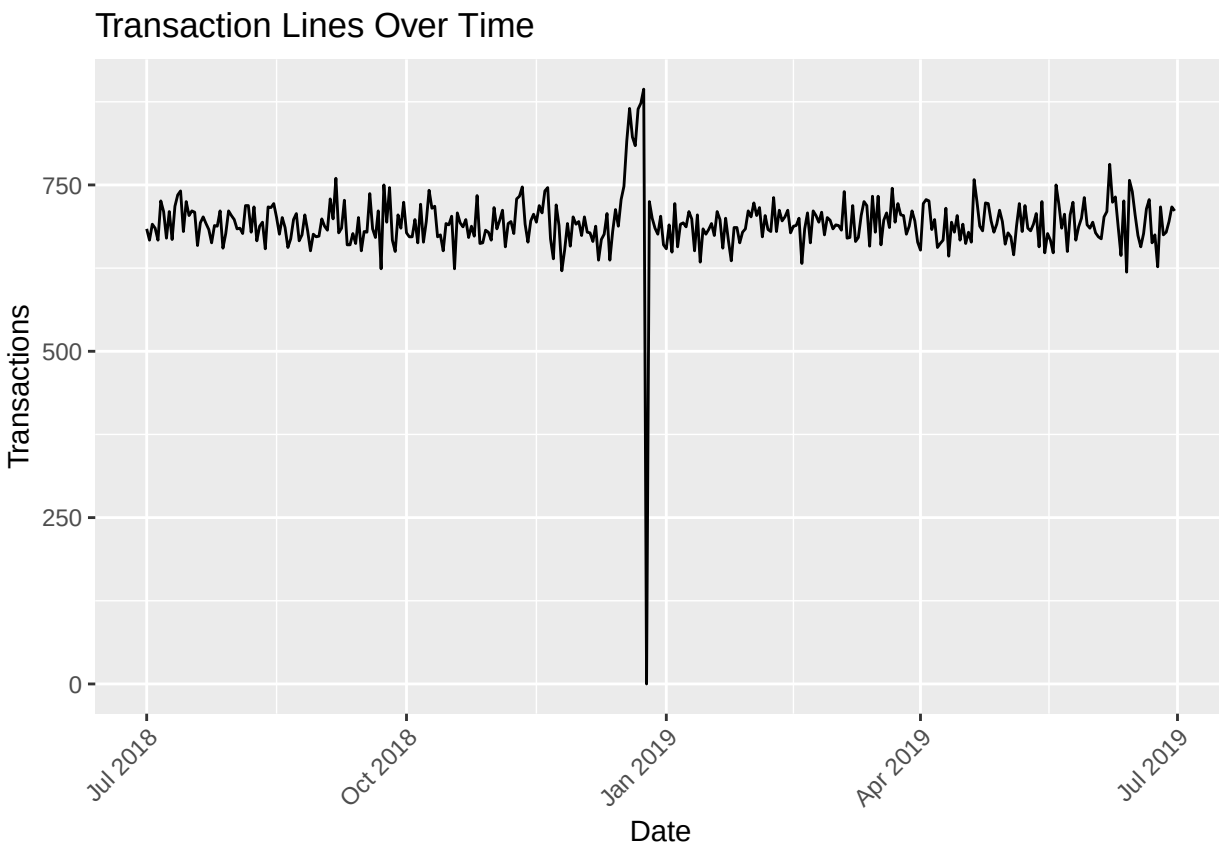
```r
#remove outlier customer quantity as is says someone purchased 200 chip packages
outlier_cards <- unique(transactionData$LYLTY_CARD_NBR[transactionData$PROD_QTY == 200])
outlier_cards
```

```
## [1] 226000
```

```r
transactionData <- transactionData[!transactionData$LYLTY_CARD_NBR %in% outlier_cards, ]


#check to see if max quantity is normal
max(transactionData$PROD_QTY)
```

```
## [1] 5
```

```r
#CHECKING DATES
# Count number of transaction lines per day
transactions_by_day <- as.data.table(transactionData)[, .N, by = DATE][order(DATE)]

# How many unique dates do we have?
nrow(transactions_by_day)

# Create a complete date sequence based on min/max in your data
all_dates <- data.table(DATE = seq(min(transactionData$DATE), max(transactionData$DATE), by = "day"))

# Join to find missing days (days with zero transactions)
transactions_by_day_full <- merge(all_dates, transactions_by_day, by = "DATE", all.x = TRUE)
transactions_by_day_full[is.na(N), N := 0]

# List missing/zero-transaction days (if any)
transactions_by_day_full[N == 0]

#result shows that Christmas 2018 is missing in this data set. this is nice to note on the report


#plot for transactions over time
ggplot(transactions_by_day_full, aes(x = DATE, y = N)) +
  geom_line() +
  labs(title = "Transaction Lines Over Time", x = "Date", y = "Transactions") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
#Create Pack_Size + Brand
#Create pack size (grams) from product name
transactionData$PACK_SIZE <- readr::parse_number(transactionData$PROD_NAME)

# create brand as first word of product name
transactionData$BRAND <- stringr::word(transactionData$PROD_NAME, 1)

#Red Rock Deli shows as RRD at times, but also shows as "RED" because taking first word of product name
transactionData$BRAND[transactionData$BRAND == "RED"] <- "RRD"

# check a few rows
head(transactionData[, c("PROD_NAME", "PACK_SIZE", "BRAND")], 5)
```

```
##                                  PROD_NAME PACK_SIZE   BRAND
##                                     <char>     <num>  <char>
## 1:   Natural Chip        Compny SeaSalt175g       175 Natural
## 2:                CCs Nacho Cheese    175g       175     CCs
## 3:   Smiths Crinkle Cut  Chips Chicken 170g       170  Smiths
## 4:   Smiths Chip Thinly  S/Cream&Onion 175g       175  Smiths
## 5: Kettle Tortilla ChpsHny&Jlpno Chili 150g       150  Kettle
```

```r
# top brands
head(sort(table(transactionData$BRAND), decreasing = TRUE), 15)
```

```
##
##    Kettle    Smiths  Pringles   Doritos     Thins       RRD Infuzions        WW
##     41288     28860     25102     24962     14075     11894     11057     10320
##      Cobs  Tostitos  Twisties  Tyrrells     Grain   Natural       Red
```

```
##      9693      9471      9454      6442      6272      6050      5885
```

```r
# Merge customer segments onto each transaction (by loyalty card number)
# For every transaction, look up the customer's segment info using the loyalty card number, and attach
data <- merge(transactionData, customerData, by = "LYLTY_CARD_NBR", all.x = TRUE)

# check that merge worked: should be same row count as transactionData
dim(transactionData)
```

```
## [1] 252589     10
```

```r
dim(data)
```

```
## [1] 252589     12
```

```r
# check for missing segment info after merge (should be 0,0)
colSums(is.na(data[, c("LIFESTAGE", "PREMIUM_CUSTOMER")]))
```

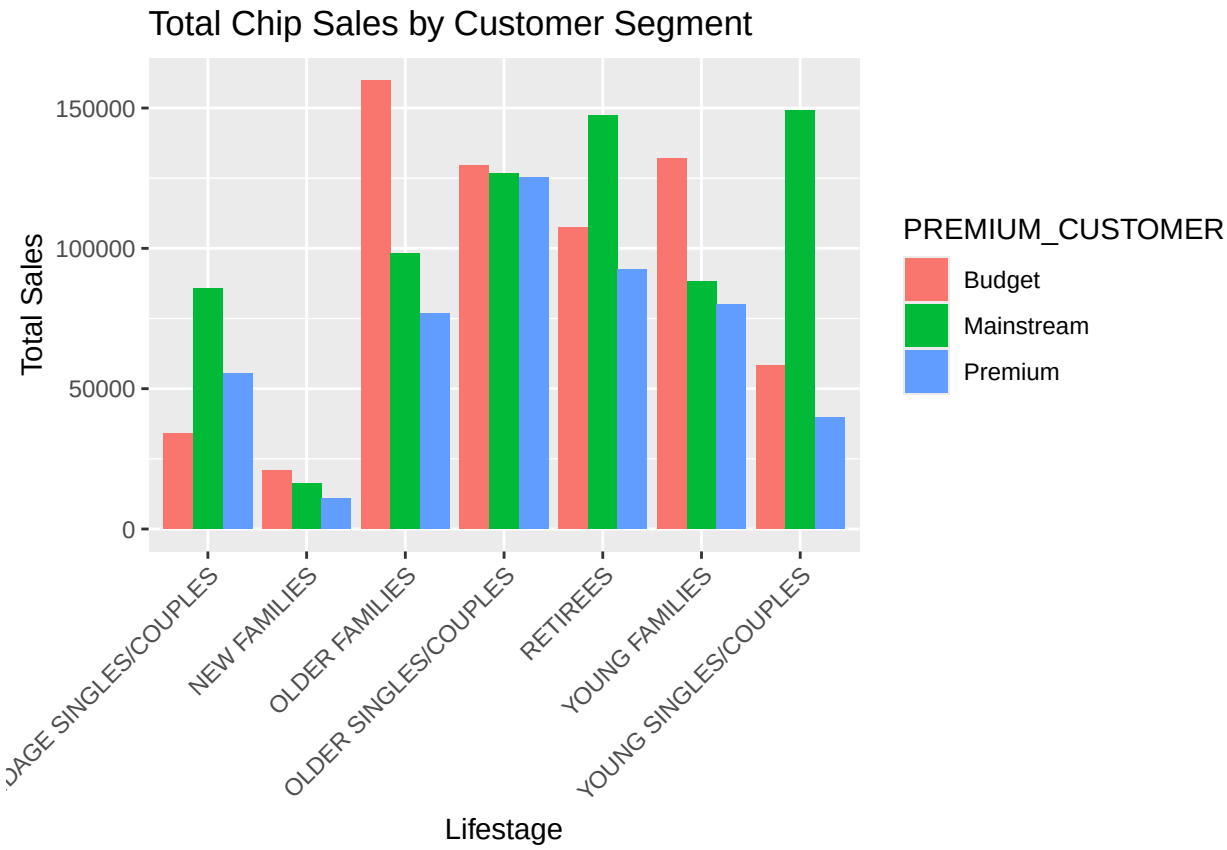
```
##        LIFESTAGE PREMIUM_CUSTOMER
##                0                0
```

```r
# Time to do analysis
# Total sales by segment
sales_by_seg <- aggregate(TOT_SALES ~ LIFESTAGE + PREMIUM_CUSTOMER, data = data, sum)

# Number of customers by segment
cust_by_seg <- aggregate(LYLTY_CARD_NBR ~ LIFESTAGE + PREMIUM_CUSTOMER, data = data,
                         function(x) length(unique(x)))
names(cust_by_seg)[3] <- "N_CUSTOMERS"

# Units per customer by segment
units_by_seg <- aggregate(PROD_QTY ~ LIFESTAGE + PREMIUM_CUSTOMER, data = data, sum)
units_by_seg <- merge(units_by_seg, cust_by_seg, by = c("LIFESTAGE", "PREMIUM_CUSTOMER"))
units_by_seg$UNITS_PER_CUST <- units_by_seg$PROD_QTY / units_by_seg$N_CUSTOMERS

# Avg price per unit by segment
price_by_seg <- aggregate(cbind(TOT_SALES, PROD_QTY) ~ LIFESTAGE + PREMIUM_CUSTOMER,
                          data = data, sum)
price_by_seg$AVG_PRICE_PER_UNIT <- price_by_seg$TOT_SALES / price_by_seg$PROD_QTY

# show top segments by total sales
sales_by_seg_sorted <- sales_by_seg[order(-sales_by_seg$TOT_SALES), ]
head(sales_by_seg_sorted, 10)
```

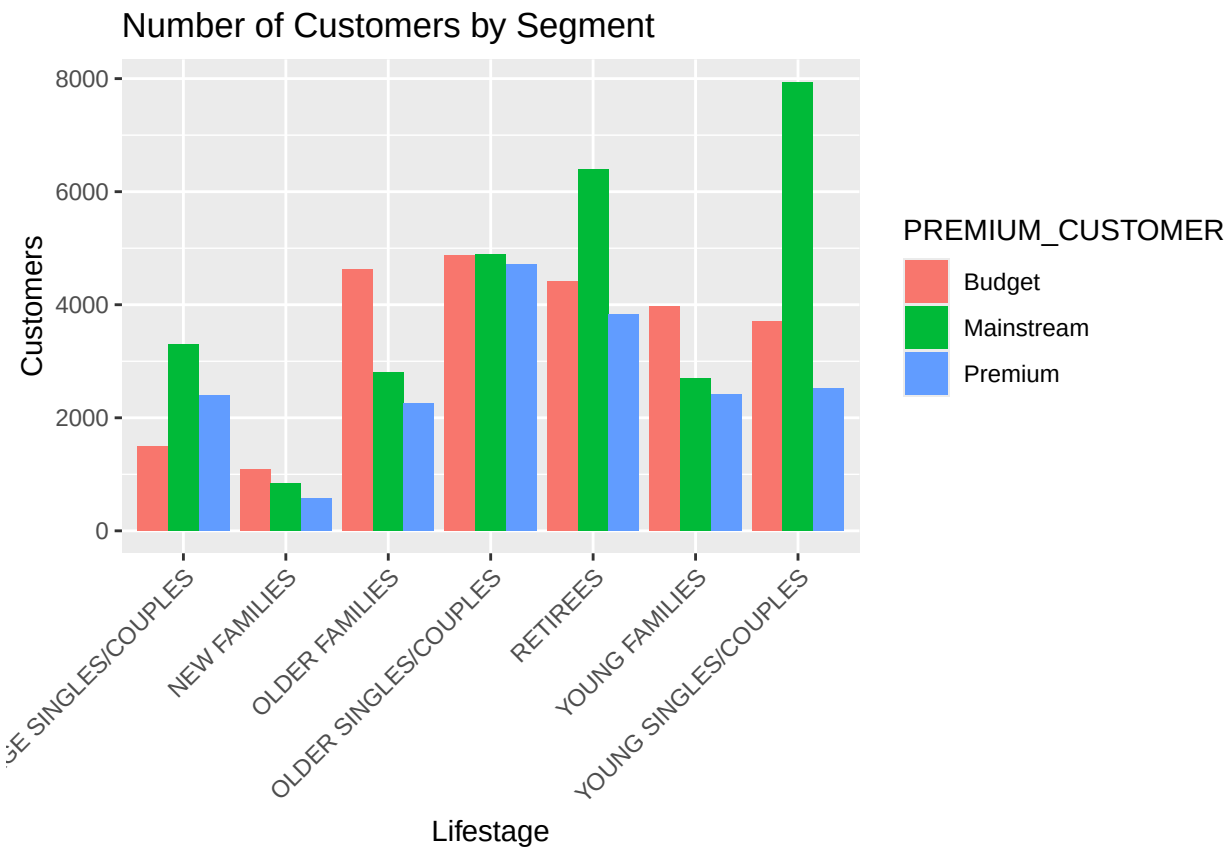
```
##                LIFESTAGE PREMIUM_CUSTOMER TOT_SALES
## 3         OLDER FAMILIES           Budget 159989.35
## 14 YOUNG SINGLES/COUPLES       Mainstream 149117.20
## 12              RETIREES       Mainstream 147444.15
## 6         YOUNG FAMILIES           Budget 132055.25
## 4  OLDER SINGLES/COUPLES           Budget 129671.80
## 11 OLDER SINGLES/COUPLES       Mainstream 126803.10
## 18 OLDER SINGLES/COUPLES          Premium 125318.45
## 5               RETIREES           Budget 107313.40
## 10        OLDER FAMILIES       Mainstream  98240.25
## 19              RETIREES          Premium  92440.75
```

```r
# plots
#total sales
```

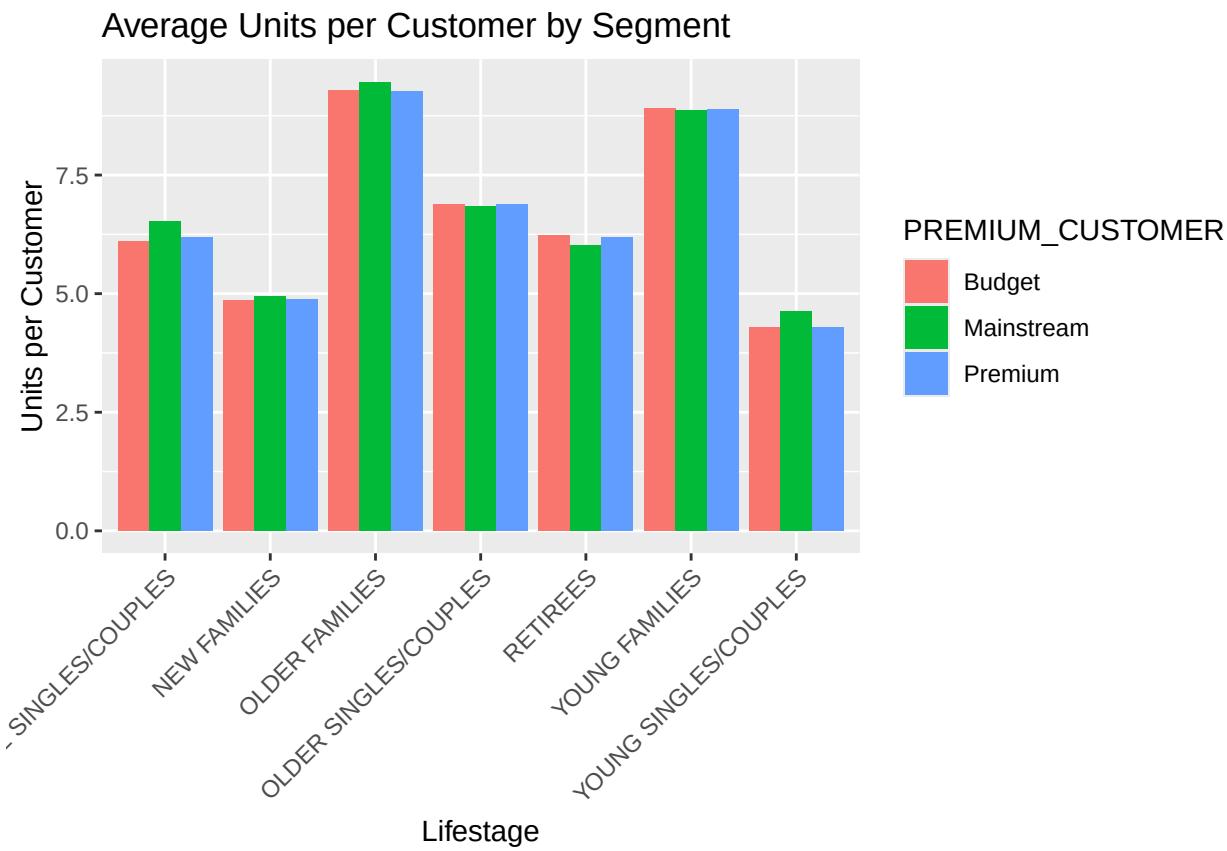
```r
ggplot(sales_by_seg, aes(x = LIFESTAGE, y = TOT_SALES, fill = PREMIUM_CUSTOMER)) +
  geom_col(position = "dodge") +
  labs(title = "Total Chip Sales by Customer Segment", x = "Lifestage", y = "Total Sales") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



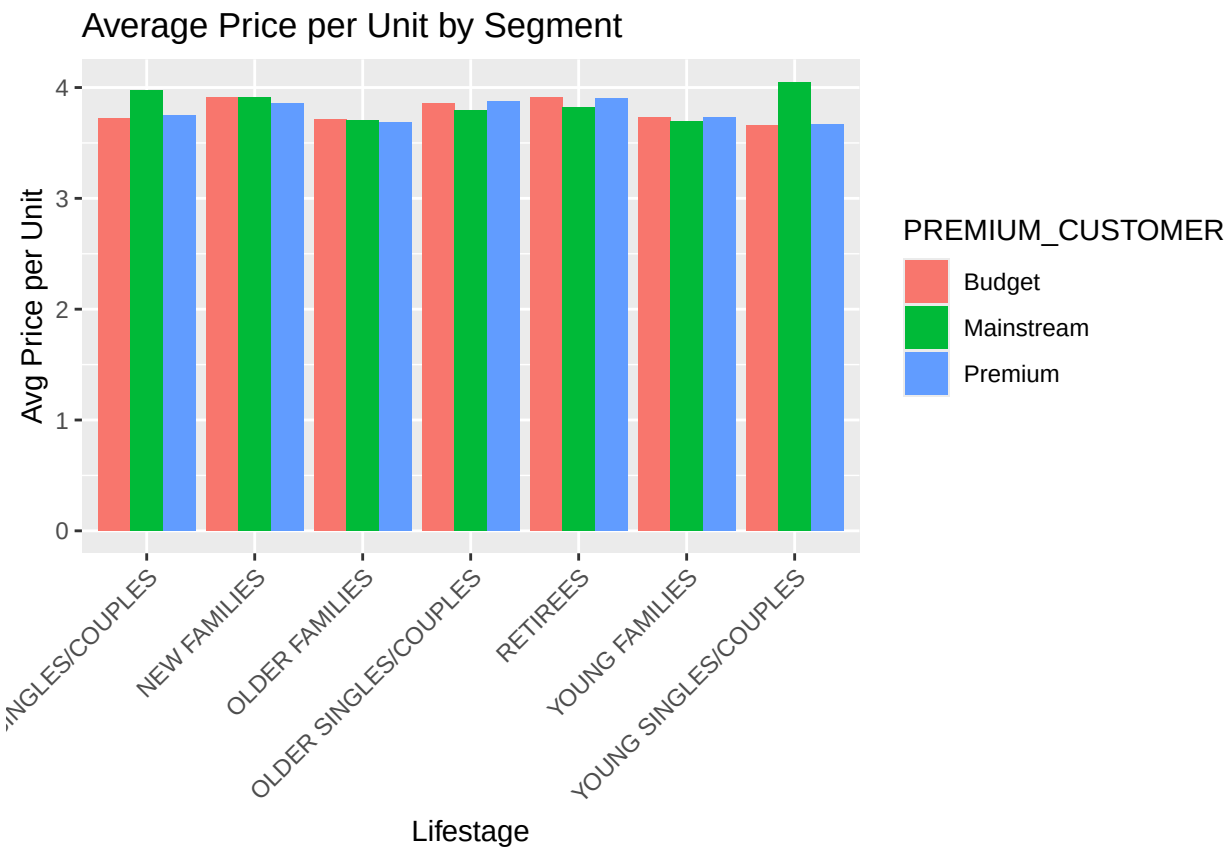

```r
#number of customers
ggplot(cust_by_seg, aes(x = LIFESTAGE, y = N_CUSTOMERS, fill = PREMIUM_CUSTOMER)) +
  geom_col(position = "dodge") +
  labs(title = "Number of Customers by Segment", x = "Lifestage", y = "Customers") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
#units per customer
ggplot(units_by_seg, aes(x = LIFESTAGE, y = UNITS_PER_CUST, fill = PREMIUM_CUSTOMER)) +
  geom_col(position = "dodge") +
  labs(title = "Average Units per Customer by Segment", x = "Lifestage", y = "Units per Customer") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
#average price per unit
ggplot(price_by_seg, aes(x = LIFESTAGE, y = AVG_PRICE_PER_UNIT, fill = PREMIUM_CUSTOMER)) +
  geom_col(position = "dodge") +
  labs(title = "Average Price per Unit by Segment", x = "Lifestage", y = "Avg Price per Unit") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
# Top 5 segments by TOTAL SALES
head(sales_by_seg_sorted, 5)
```

```
##                LIFESTAGE PREMIUM_CUSTOMER TOT_SALES
## 3         OLDER FAMILIES           Budget  159989.4
## 14 YOUNG SINGLES/COUPLES       Mainstream  149117.2
## 12              RETIREES       Mainstream  147444.1
## 6         YOUNG FAMILIES           Budget  132055.2
## 4  OLDER SINGLES/COUPLES           Budget  129671.8
```

```r
# Top 5 segments by NUMBER OF CUSTOMERS
cust_by_seg_sorted <- cust_by_seg[order(-cust_by_seg$N_CUSTOMERS), ]
head(cust_by_seg_sorted, 5)
```

```
##                LIFESTAGE PREMIUM_CUSTOMER N_CUSTOMERS
## 14 YOUNG SINGLES/COUPLES       Mainstream        7943
## 12              RETIREES       Mainstream        6400
## 11 OLDER SINGLES/COUPLES       Mainstream        4883
## 4  OLDER SINGLES/COUPLES           Budget        4878
## 18 OLDER SINGLES/COUPLES          Premium        4709
```

```r
# Top 5 segments by UNITS PER CUSTOMER
units_by_seg_sorted <- units_by_seg[order(-units_by_seg$UNITS_PER_CUST), ]
head(units_by_seg_sorted, 5)
```

```
##         LIFESTAGE PREMIUM_CUSTOMER PROD_QTY N_CUSTOMERS UNITS_PER_CUST
## 8  OLDER FAMILIES       Mainstream    26501        2800       9.464643
## 7  OLDER FAMILIES           Budget    43044        4635       9.286731
## 9  OLDER FAMILIES          Premium    20795        2247       9.254562
```

```
## 16 YOUNG FAMILIES           Budget    35373        3968       8.914567
## 18 YOUNG FAMILIES          Premium    21398        2411       8.875156
```

```r
# Top 5 segments by AVG PRICE PER UNIT
price_by_seg_sorted <- price_by_seg[order(-price_by_seg$AVG_PRICE_PER_UNIT), ]
head(price_by_seg_sorted, 5)
```

```
##                 LIFESTAGE PREMIUM_CUSTOMER TOT_SALES PROD_QTY
## 14  YOUNG SINGLES/COUPLES       Mainstream 149117.20    36809
## 8  MIDAGE SINGLES/COUPLES       Mainstream  85665.75    21568
## 9            NEW FAMILIES       Mainstream  16158.60     4128
## 2            NEW FAMILIES           Budget  20838.25     5329
## 5                RETIREES           Budget 107313.40    27464
##    AVG_PRICE_PER_UNIT
## 14           4.051107
## 8            3.971891
## 9            3.914390
## 2            3.910349
## 5            3.907421
```

Key insights

- Sales are highest for Older Families (Budget), Young Singles/Couples (Mainstream), and Retirees (Mainstream).
- Young Singles/Couples (Mainstream) and Retirees (Mainstream) have the most customers, which helps explain their high sales.
- Family segments (Older Families, Young Families) buy the most units per customer, so their sales are more volume-driven.
- Mainstream singles/couples pay a higher average price per unit, suggesting a willingness to pay for preferred brands or flavors.

Recommendation to Julia

- Focus growth efforts on Older Families (Budget) with volume offers (multi-buy, larger packs) since they purchase more units per customer.
- For Young Singles/Couples (Mainstream), prioritize popular brands and flavor variety and run occasion based promotions (weekends, gatherings) since this segment has high customer count and higher price per unit.
- Maintain core range availability for Retirees (Mainstream) to protect a large customer base and steady sales.